

# SB Stocks

## *A MERN Stack Stock Trading Simulation Platform*

|                  |                                               |
|------------------|-----------------------------------------------|
| Project Title    | SB Stocks – Stock Trading Application         |
| Date             | 17 July 2026                                  |
| Technology Stack | MongoDB, Express.js, React.js, Node.js (MERN) |

## 1. Introduction

### Project Title

SB Stocks – Stock Trading Application

## 2. Project Overview

### Purpose

SB Stocks is a simulated stock trading platform that lets a user practice buying and selling equities with virtual money in a realistic, real-time environment. It gives traders a risk-free space to learn how markets, order execution, and portfolio tracking work, backed by a live-feeling ticker engine, an interactive dashboard, and a full trade/wallet audit trail — without touching real capital.

### Features

- User registration and secure login (JWT-based authentication)
- Real-time market dashboard with a 4-second auto-refreshing ticker engine
- Top Gainers / Top Losers widgets and stock search
- Interactive stock detail page with Candlestick, Glowing Trend, and Volume & Spread chart views across 1D/1W/1M/1Y ranges
- Buy and Sell order execution at live market price, with Intraday/Delivery product types
- Portfolio management — holdings, average purchase price, current value, and profit/loss
- Portfolio weighting visualization (donut chart)
- Wallet funding simulator — deposit and withdraw virtual funds
- Trading Book & Audit History — full order book and wallet ledger with timestamps
- User profile management with role-based access (User / Admin)

## 3. Architecture

### Frontend

The client is a React.js single-page application. Pages (Login/Register, Dashboard, Stock Detail, Portfolio, History, Profile) are composed from reusable components and routed with React Router. Tailwind CSS handles styling with a dark-themed UI, and an Axios instance centralizes calls to the backend REST API. Live prices are reflected on-screen by polling/subscribing to the ticker engine so the dashboard and stock detail charts stay current without a manual refresh.

## Backend

The server is built on Node.js with Express.js, exposing a REST API for authentication, market data, trading, wallet, and history. A JWT authentication middleware protects all routes except registration and login. A lightweight ticker engine runs on the server, updating simulated stock prices at a fixed interval and persisting them so every connected client sees consistent live data. Business rules — such as rejecting a buy order that exceeds available cash, or a sell order that exceeds owned shares — are enforced in the controllers before any database write.

## Database

MongoDB (via Mongoose) stores all application data in five main collections:

| Collection   | Key Fields                                                                                                     |
|--------------|----------------------------------------------------------------------------------------------------------------|
| users        | id, username, email, passwordHash, role (User/Admin), walletBalance, createdAt                                 |
| stocks       | id, symbol, companyName, currentPrice, changePercent, exchange, priceHistory[]                                 |
| orders       | _id, userId, symbol, orderType (BUY/SELL), productType, quantity, pricePerShare, totalPrice, status, timestamp |
| holdings     | id, userId, symbol, shares, avgPurchasePrice                                                                   |
| walletledger | id, userId, type (DEPOSIT/WITHDRAW), amount, fundingSource, timestamp                                          |

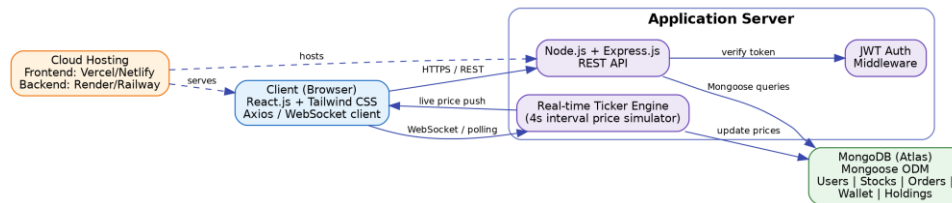


Figure 1: SB Stocks technical architecture (MERN stack)

## 4. Setup Instructions

### Prerequisites

- Node.js (v18 or later) and npm
- MongoDB — a local instance or a free MongoDB Atlas cluster
- Git
- A code editor such as VS Code

### Installation

1. Clone the repository:

```
git clone <repository-url>
cd sb-stocks
```

2. Install backend dependencies:

```
cd server
npm install
```

3. Install frontend dependencies:

```
cd ../client
npm install
```

4. Create a .env file inside the server directory with the following environment variables:

```
PORT=5000
MONGO_URI=<your MongoDB connection string>
JWT_SECRET=<your JWT secret>
CLIENT_URL=http://localhost:3000
```

5. Create a .env file inside the client directory pointing to the API:

```
REACT_APP_API_URL=http://localhost:5000/api
```

## 5. Folder Structure

### Client (React frontend)

```
client/
├── public/
├── src/
│   ├── components/      # Navbar, StockRow, TradePanel, PortfolioChart, ...
│   ├── pages/           # Login, Register, Dashboard, StockDetail,
│                       # Portfolio, History, Profile
│   ├── context/         # AuthContext (JWT/session state)
│   ├── services/        # api.js - Axios instance & API calls
│   ├── App.js           # Route definitions
│   └── index.js
└── package.json
```

### Server (Node.js backend)

```
server/
├── config/              # db.js - MongoDB connection
├── models/              # User.js, Stock.js, Order.js,
│                       # Holding.js, WalletLedger.js
├── controllers/         # authController.js, tradeController.js,
│                       # walletController.js, portfolioController.js
├── routes/              # authRoutes.js, stockRoutes.js, tradeRoutes.js,
│                       # walletRoutes.js, userRoutes.js
├── middleware/          # authMiddleware.js - JWT verification
├── utils/               # tickerEngine.js - real-time price simulator
├── server.js            # App entry point
└── package.json
```

## 6. Running the Application

Start the backend server:

```
cd server
npm start
# Server runs on http://localhost:5000
```

Start the frontend (in a separate terminal):

```
cd client
npm start
# Client runs on http://localhost:3000
```

## 7. API Documentation

The base URL for all endpoints is /api. Protected endpoints require an Authorization: Bearer <token> header obtained from the login response.

| Method | Endpoint             | Description                                         | Auth |
|--------|----------------------|-----------------------------------------------------|------|
| POST   | /api/auth/register   | Register a new user (name, email, password)         | No   |
| POST   | /api/auth/login      | Authenticate a user and issue a JWT                 | No   |
| GET    | /api/stocks          | Get the live list of stocks with price and % change | Yes  |
| GET    | /api/stocks/:symbol  | Get chart/price-history data for one stock          | Yes  |
| POST   | /api/trade/buy       | Execute a buy order (symbol, quantity, productType) | Yes  |
| POST   | /api/trade/sell      | Execute a sell order (symbol, quantity)             | Yes  |
| GET    | /api/portfolio       | Get current holdings, valuation and profit/loss     | Yes  |
| GET    | /api/orders          | Get the executed order book (trade history)         | Yes  |
| GET    | /api/wallet          | Get wallet balance and transaction ledger           | Yes  |
| POST   | /api/wallet/deposit  | Deposit virtual funds into the wallet               | Yes  |
| POST   | /api/wallet/withdraw | Withdraw virtual funds from the wallet              | Yes  |
| GET    | /api/profile         | Get the logged-in user's profile details            | Yes  |

Example — POST /api/auth/login

```
// Request body
{ "email": "karthik555@example.com", "password": "*****" }

// Response
{ "token": "<jwt>", "user": { "username": "karthik555", "role": "USER" } }
```

Example — POST /api/trade/buy

```
// Request body
{ "symbol": "AAPL", "quantity": 1, "productType": "Intraday" }

// Response
{ "orderId": "...", "status": "Completed", "totalPrice": 472.85 }
```

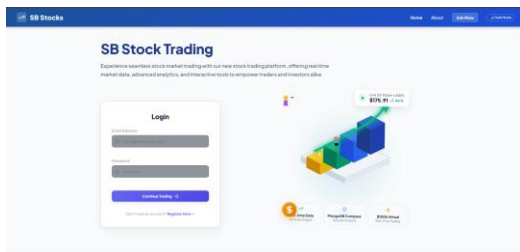
## 8. Authentication

SB Stocks uses stateless JWT-based authentication. On registration, the password is hashed with bcrypt before being stored — plain-text passwords are never persisted. On login, the server verifies the credentials and issues a signed JWT containing the user ID and role.

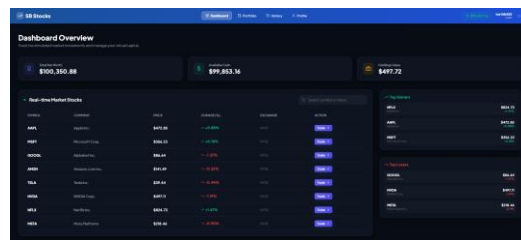
- The client stores the JWT and attaches it as a Bearer token on every subsequent API request.
- An authMiddleware verifies the token's signature and expiry on protected routes, and attaches the authenticated user to the request.
- A role field (User / Admin) on the user document enables role-based authorization for future admin-only endpoints.
- Requests with a missing, invalid, or expired token are rejected with a 401 Unauthorized response.

## 9. User Interface

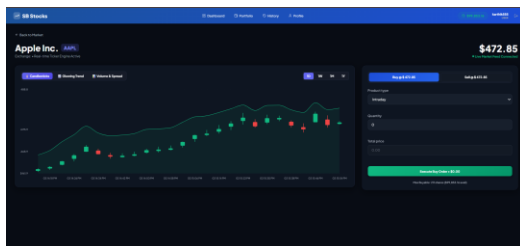
Screenshots of the core screens are shown below, two per row.



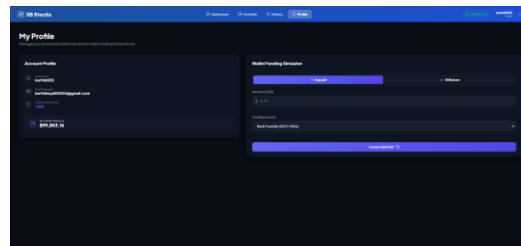
*Login / Registration*



*Dashboard Overview*



*Stock Detail — Candlestick chart & Buy/Sell panel*



*My Profile & Wallet Funding*

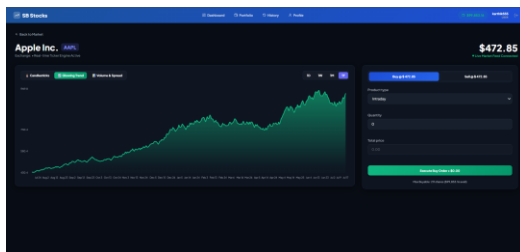
## 10. Testing

Testing during development focused on functional and manual verification of each trading flow:

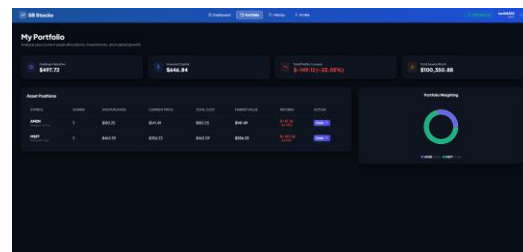
- Manual end-to-end testing of registration, login, buy/sell execution, deposits/withdrawals, and history logging through the UI.
- API endpoints were exercised individually with Postman to verify request validation, auth enforcement, and response shapes.
- Edge cases checked: buying with insufficient funds, selling more shares than owned, and invalid/expired JWTs.
- Planned automated coverage: Jest + React Testing Library for frontend components, and Jest + Supertest for backend route/integration tests.

## 11. Screenshots or Demo

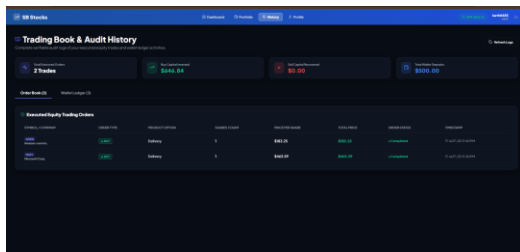
The screenshots below demonstrate the core trading and reporting flow.



*Stock Detail — 1Y trend view*



*My Portfolio*



*Trading Book & Audit History*

*A recorded demo walkthrough can be linked here: [Add demo video / hosted app link]*

## 12. Known Issues

- Stock prices are generated by an internal simulator rather than sourced from a live market-data provider.
- No real payment gateway is integrated — wallet funding is a virtual/simulated transaction only.
- Email/OTP confirmation on registration is not yet wired to a live mail provider.
- No automated test suite is currently included in the repository.
- UI has not been fully optimized for very small mobile screens.

## 13. Future Enhancements

- Integrate a real market-data API for live prices in place of the simulator.
- Add an admin dashboard for monitoring users, trades, and platform-wide activity.
- Email/OTP-based registration confirmation via a transactional email service.
- Watchlists and price alerts for tracked stocks.
- Support for additional instruments such as mutual funds or options.
- Automated test coverage (Jest, React Testing Library, Supertest) and CI pipeline.
- Native mobile app or a fully responsive mobile-first redesign.